Multi-agent systems module · Group project

Centralized Multi-Agent Shopping Assistant

Requirements and Implementation Specification

Document	Software requirements specification and implementation guide
System	Centralized multi-agent shopping assistant website (food, clothes, electronics)
Version	1.0 (merges all revisions agreed so far)
Date	10 October 2026
Status	Draft for team review

This document describes a website where registered customers ask questions about food, clothes and electronics products in plain language. A central router agent splits each question by category, sends the parts to specialist agents and combines their answers into one reply. Each specialist agent keeps its own database, filled automatically every day by scraping its own list of shopping websites. Every user's chat history is saved, and administrators can review it.

What this document covers

- The problem and the proposed multi-agent solution (Sections 2–3)
- Users, roles, and functional and non-functional requirements (Sections 4–7)
- Data model, process flows and interfaces (Sections 8–10)
- A step-by-step implementation guide with code skeletons (Section 11)
- Test cases, traceability, risks and open questions (Sections 12–14)

Document history

Version	Changes
0.1	Initial architecture and requirements from the team's concept sketch.
0.2	Added user management, permanently saved chat history, admin access and combined answers for multi-category questions.
0.3	Follow-ups after a combined answer now ask which category; the chatbot is shown only to logged-in users.
1.0	Product images removed from the databases. All revisions merged into this single document, with implementation guide, test plan and traceability.

Contents

1. Introduction . . . 5

1.1 Purpose . . . 5

1.2 Scope . . . 5

1.3 Definitions and abbreviations . . . 5

1.4 Original concept . . . 6

2. Problem statement . . . 7

2.1 Background . . . 7

2.2 Problem . . . 7

2.3 Objectives . . . 7

2.4 Success criteria . . . 7

3. Proposed solution . . . 8

3.1 Overview . . . 8

3.2 System architecture . . . 9

3.3 Components . . . 10

3.4 Agent design . . . 10

3.5 Key design decisions . . . 11

4. Stakeholders and user roles . . . 12

4.1 Stakeholders . . . 12

4.2 Roles and permissions . . . 12

5. Functional requirements . . . 13

5.1 Summary of functional requirements . . . 13

5.2 Chat and conversation history . . . 14

5.3 Understanding and routing . . . 14

5.4 Combined answers . . . 15

5.5 Data collection and storage . . . 16
5.6 Logging . . . 16
5.7 User management . . . 17

6. Non-functional requirements . . . 19

7. Constraints, assumptions and scope . . . 20

7.1 Constraints . . . 20
7.2 Assumptions . . . 20
7.3 Out of scope . . . 20

8. Data requirements . . . 21

8.1 Product data (food.db, clothes.db, electronics.db) . . . 21
8.2 Application data (app.db) . . . 22
8.3 Retention and deletion . . . 23
8.4 Website list configuration . . . 24

9. System behaviour . . . 25

9.1 Registration and login . . . 25
9.2 Single-category question . . . 25
9.3 Multi-category question (combined answer) . . . 26
9.4 Follow-up after a combined answer . . . 27
9.5 Daily data refresh . . . 28
9.6 Admin reviews a user's history . . . 28
9.7 Error handling . . . 29

10. Interface requirements . . . 30

10.1 User interface pages . . . 30
10.2 API endpoints . . . 30
10.3 Internal agent interfaces . . . 31

11. Implementation guide . . . 32

11.1 Technology stack . . . 32
11.2 Project structure . . . 32
11.3 Core data structures (app/db/models.py) . . . 34
11.4 Domain agents (app/agents/) . . . 35
11.5 Central router agent (app/orchestrator/central_agent.py) . . . 36
11.6 Task planner (app/orchestrator/task_planner.py) . . . 37
11.7 Answer combiner (app/orchestrator/answer_combiner.py) . . . 37
11.8 Scrapers (app/scrapers/) . . . 38
11.9 Authentication and access control (app/auth/) . . . 38

11.10 Frontend login guard (frontend/js/chat.js) . . . 39
11.11 Scheduler and application start-up . . . 40
11.12 Configuration and running locally . . . 40
11.13 Build plan and team responsibilities . . . 41

12. Testing and acceptance . . . 42
12.1 Test strategy . . . 42
12.2 Test cases . . . 42
12.3 Requirements traceability . . . 44
12.4 Acceptance . . . 45

13. Risks and mitigations . . . 46

14. Open questions . . . 47

Appendix A. Demo script . . . 48

1. Introduction

1.1 Purpose

This document defines **what** the Centralized Multi-Agent Shopping Assistant must do (its requirements) and **how** the team will build it (the implementation guide). It merges every design decision agreed so far into one reference for the development team, reviewers and the module lecturer. Where a decision is still open, the document states the default the team will build until the question is settled (Section 14).

1.2 Scope

The system is a website. Registered customers log in and ask natural-language questions about food, clothes and electronics products. A central router agent sends each question (or each part of a multi-category question) to the matching specialist agent. Each specialist agent answers only from its own database, which it refreshes every day by scraping its own list of shopping websites. The system saves each user's chat history and lets administrators review any user's history.

The system covers:

- Customer registration, login, roles and account management
- A chat interface, shown only to logged-in users, with permanently saved conversations
- Category detection, routing, and combined answers for multi-category questions
- Three domain agents (food, clothes, electronics), each with its own website list and database
- A daily automatic scrape that stores text-only product data (no images)
- An admin dashboard for reviewing users and their chat histories

Items that are explicitly out of scope are listed in Section 7.3.

1.3 Definitions and abbreviations

Term	Meaning
Agent	A software component that perceives its environment and acts on its own to reach a goal.
Central router agent	The single agent that receives every customer message, splits it into tasks, routes the tasks to domain agents and combines their answers. Also called the central agent.
Domain agent	A specialist agent for one product category: food, clothes or electronics.
Task	One sub-question for one category, created by the task planner, e.g. {domain: clothes, query: “rain jacket”}.
Task planner	The part of the central agent that detects categories and creates tasks.
Answer combiner	The part of the central agent that merges agent results into one reply.
Scraping	Automatically reading product information from public web pages.
Upsert	Insert a row if it is new, or update it if it already exists.
Conversation	A thread of messages between one user and the assistant.
JWT	JSON Web Token: a signed login token sent with every request.
LLM	Large language model, used to understand questions and write answers.
PEAS	Performance measure, Environment, Actuators, Sensors: a way to describe an agent.
CSP	Constraint satisfaction problem.

Term	Meaning
MoSCoW	Priority scheme: Must have, Should have, Could have, Won't have.
Guest	A visitor who has not logged in.

1.4 Original concept

The design started from the team's hand-drawn sketch (Figure 1). The table below records how each element of the sketch was interpreted and how later decisions changed it.

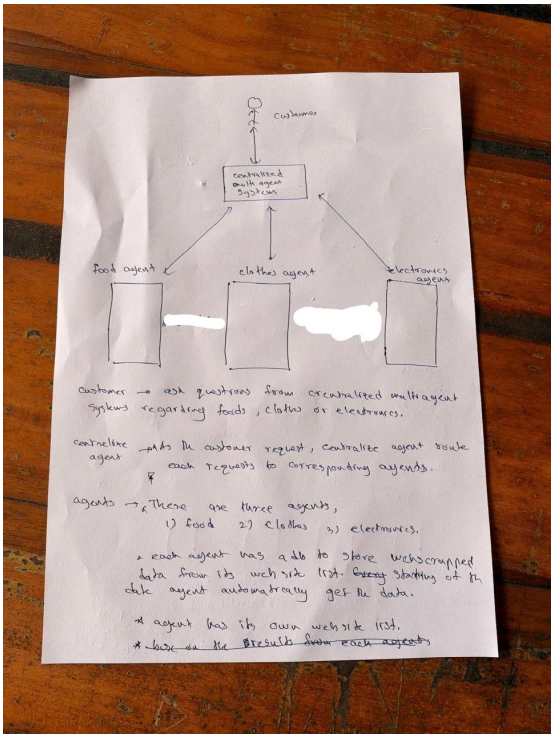


Figure 1. The team's original concept sketch

Element in the sketch	Interpretation in this document
Customer ↔ centralized multi-agent system	Customers ask questions in natural language through the website.
Central box	The central router agent: receives every request and routes it to the right domain agent or agents.
Food, clothes and electronics agents, each with a box	Three domain agents, each with its own database (the boxes).
Note: each agent has a DB of web-scraped data from its website list	Each agent owns a website list and a product database (FR-5, FR-6).
Note: at the start of each day the agent gets the data automatically	Automatic daily refresh at 00:00 (FR-7).
Struck-out note: “based on the results from each agent”	Combining results was first dropped, then reinstated as combined answers (FR-11).
Whited-out areas between the agent boxes	Unreadable. This document assumes agents do not talk to each other directly (A-1, OQ-1).
Not in the sketch; added by the team	User management, saved chat history, admin access, clarification rules, no stored images.

2. Problem statement

2.1 Background

People shopping online for groceries, clothing and electronics usually have to visit several websites, search each one separately and compare the results by hand. Product information is spread across sites with different layouts, search tools and category names. A customer who needs items from more than one category, for example a rain jacket and a power bank, has to repeat the whole process for each category. Earlier searches are lost unless the customer keeps their own notes.

2.2 Problem

There is no single place where a customer can ask, in plain language, about products from several categories and several websites, get one clear answer with links to the source websites, and return later to their previous conversations.

2.3 Objectives

ID	Objective
O-1	Provide one chat interface for food, clothes and electronics questions.
O-2	Route each question automatically to the right specialist agent or agents.
O-3	Combine the answers for multi-category questions into one reply.
O-4	Keep product data fresh with an automatic daily scrape for each agent.
O-5	Save every user's chat history and let administrators review it.
O-6	Demonstrate core multi-agent concepts: specialization, coordination by a central agent, autonomy (daily self-refresh), encapsulated data and fault isolation.

2.4 Success criteria

- At least 90% of the labelled routing test set is routed to the correct category or categories.
- Every answer contains only products from the databases, each with a source link.
- Single-category answers arrive in 10 seconds or less, and combined answers in 15 seconds or less, for 95% of requests on demo hardware.
- The daily refresh runs without manual action for 7 consecutive days during testing.
- All access-control test cases in Section 12 pass.

3. Proposed solution

3.1 Overview

The solution is a **centralized multi-agent system** (supervisor pattern). One central router agent is the only entry point for customer messages. Behind it are three independent domain agents (food, clothes and electronics), each owning its own website list, scraper and database. A user management service handles registration, login, roles and chat history, which are stored in a separate application database, app.db.

When a customer sends a message, the central agent's task planner works out which categories the message is about and creates one task per category. The tasks go to the matching domain agents at the same time. Each domain agent searches its own database and returns matching products. The answer combiner merges the results into one reply with a section per category, and the exchange is saved to the customer's history.

Separately from customer requests, a scheduler starts every domain agent's refresh at the start of each day. The agent scrapes its websites and updates its database with text-only product data.

3.2 System architecture

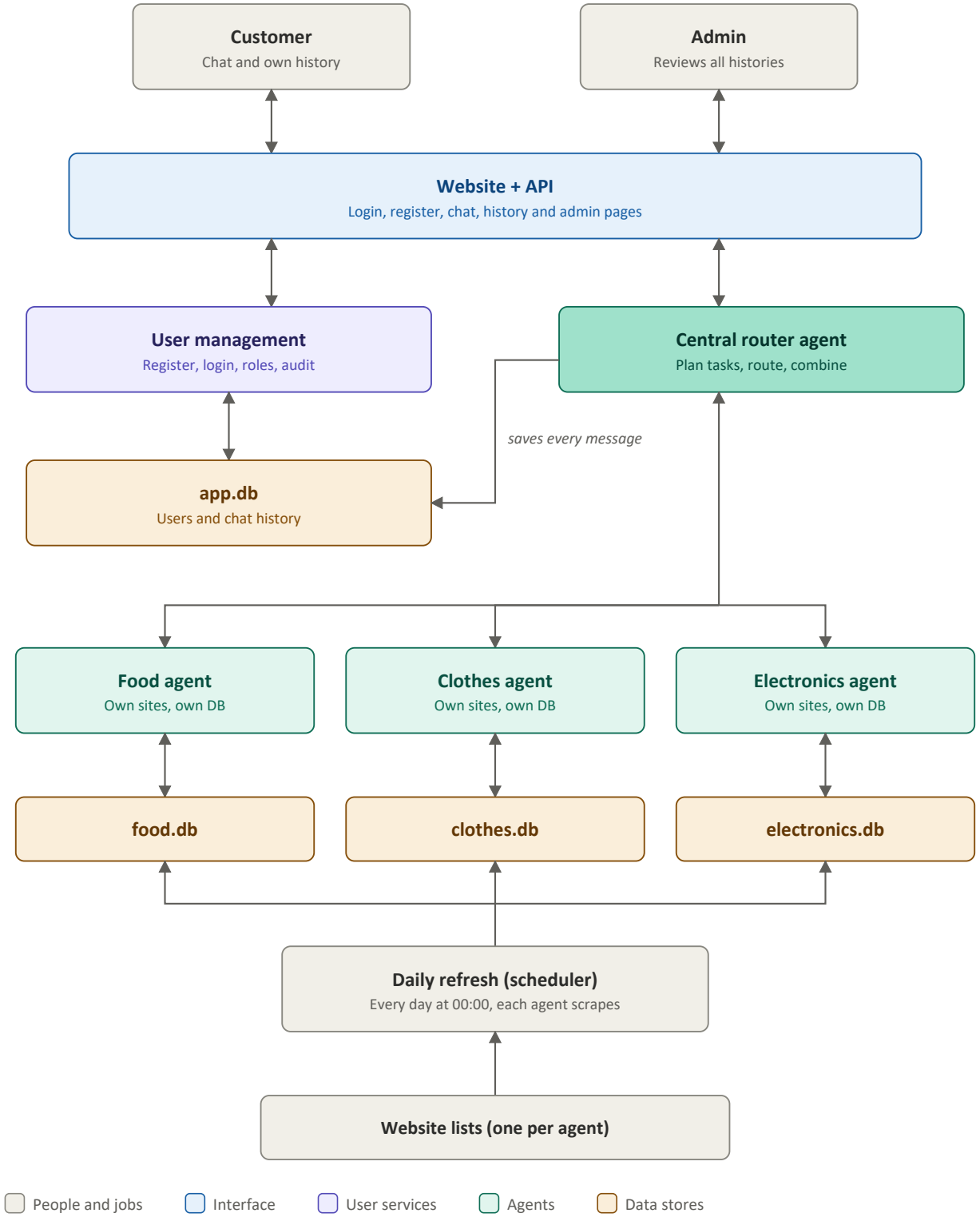


Figure 2. System architecture. Double-headed arrows are request and response pairs.

3.3 Components

Component	Responsibility	Technology
Website (frontend)	Login and registration pages; chat page with past-conversations sidebar; admin dashboard. The chatbot is shown only to logged-in users.	HTML, CSS, JavaScript
API layer	HTTP endpoints, token checks, passes chat messages to the central agent.	FastAPI
User management	Registration, login, password hashing, roles, account activation, audit of admin access.	FastAPI module, bcrypt, PyJWT
app.db	Users, conversations, messages, agent calls, admin audit.	SQLite + SQLAlchemy
Central router agent	Task planner (detect categories, create tasks), parallel dispatch, answer combiner, clarifying questions, saving history.	Python asyncio, LLM
Domain agents (3)	Search own database, write a grounded answer for their category, refresh own data daily.	Python, LLM
Product databases (3)	food.db, clothes.db and electronics.db: text-only product data and scrape history.	SQLite
Scrapers (3)	Fetch and parse each agent's websites; normalize to the product schema.	requests, BeautifulSoup, Playwright
Scheduler	Starts each agent's refresh at 00:00 every day, as separate jobs.	APScheduler
LLM client	One wrapper around the chosen LLM provider.	Python

3.4 Agent design

The agents are described below using the PEAS framework.

	Central router agent	Domain agent (food, clothes, electronics)
Performance measure	Correct routing and splitting; clarifying only when needed; every sub-question answered; reply time.	Relevance of returned products; accuracy of prices and links; data freshness; scrape success rate.
Environment	Customer messages, conversation history, the three domain agents.	Its own website list, its own product database, tasks from the central agent.
Actuators	Sends tasks to agents; sends replies and clarifying questions; writes chat history.	HTTP requests to websites; writes to its database; replies to the central agent.
Sensors	Incoming message, stored history, agent replies.	Web page content, tasks from the central agent, database query results, scheduler trigger.
Agent type	Goal-based agent with internal state (conversation history).	Model-based agent: keeps an internal model of its domain (the product database), updated daily; goal-based when answering a task.

Environment properties

Property	Central router agent	Domain agent
Observable	Partially: customer intent must be inferred	Partially: sees only its listed websites
Single or multi-agent	Multi-agent	Multi-agent
Deterministic	Stochastic: customer input is unpredictable	Stochastic: website content changes

Property	Central router agent	Domain agent
Episodic or sequential	Sequential: follow-ups depend on history	Episodic: each task is independent
Static or dynamic	Dynamic: must reply within a time limit	Dynamic: data changes daily
Discrete or continuous	Discrete	Discrete

Multi-agent characteristics shown by the system

- **Specialization:** each domain agent knows only its own category and data.
- **Coordination:** the central agent plans, delegates and combines; agents do not coordinate among themselves.
- **Autonomy:** every domain agent refreshes its own knowledge daily without being asked.
- **Encapsulation:** each agent owns its database; no agent can read another's data or user data.
- **Fault isolation:** one failing agent or website never stops the others.
- **Extensibility:** a fourth category (e.g. books) can be added without changing existing agents.

3.5 Key design decisions

Topic	Decision	Reason
Coordination model	Centralized (supervisor) pattern	Matches the sketch; single entry point; easy to test and explain.
Agent-to-agent communication	None; everything goes through the central agent	Keeps agents independent (A-1, OQ-1).
Data ownership	One database per domain agent plus a separate app.db	Matches the sketch; isolation; agents never see user data.
Multi-category questions	Split into tasks, run in parallel, combine into one reply	Team decision, v0.2.
Ambiguous follow-up after a combined answer	Ask which category	Team decision, v0.3; avoids guessing.
Access to the chatbot	Login required; chatbot hidden from guests	Team decision, v0.3; history must belong to a user.
Product images	Not stored (no image URLs or files)	Team decision, v1.0; smaller databases and lighter scraping.
Data refresh	Daily at 00:00, one job per agent	Sketch: “start of the day”; jobs isolated.
LLM provider	Any, behind one wrapper	Lets the team use whichever provider or local model is available.

4. Stakeholders and user roles

4.1 Stakeholders

Stakeholder	Interest
Customer	Quick, accurate product answers across categories; one combined reply; saved history.
Administrator	Monitor usage, review chat histories, manage accounts.
Development team	Build, test and maintain the system; demonstrate multi-agent design.
Module lecturer / examiner	Evaluate the multi-agent design, implementation and documentation.
Website owners	Their public product pages are scraped; expect respectful scraping (robots.txt, low request rate).

4.2 Roles and permissions

The system has two roles, **customer** and **admin**. Visitors who have not logged in are guests.

Action	Guest	Customer	Admin
See the login and registration pages	Yes	Yes	Yes
Register a customer account	Yes	–	–
Use the chatbot	No	Yes	Yes
List, open, continue and delete own conversations	No	Yes	Yes
View other users' conversations (read-only, audited)	No	No	Yes
List and search users	No	No	Yes
Deactivate or reactivate accounts	No	No	Yes
Create admin accounts	No	No	Setup script only

5. Functional requirements

Priorities use MoSCoW: **Must** is required for the system to be accepted, **Should** is expected but not blocking, and **Could** is a stretch goal. Requirement IDs are stable and are referenced by the test cases in Section 12.

5.1 Summary of functional requirements

ID	Requirement	Priority
FR-1	Customer chat with saved history	Must
FR-2	Category detection and routing	Must
FR-3	Unclear, out-of-scope and follow-up messages	Must
FR-4	Grounded answers	Must
FR-5	Website list per agent	Must
FR-6	Separate database per agent	Must
FR-7	Automatic daily refresh	Must
FR-8	Common product format, without images	Must
FR-9	Data freshness shown	Should
FR-10	Logging and traceability	Should
FR-11	Combined reply for multi-category questions	Must
FR-12	Partial failure handling	Must
FR-13	No invented content in combined replies	Must
FR-14	Shared constraints across categories (stretch goal)	Could
UM-1	Registration	Must
UM-2	Login and logout	Must
UM-3	Roles	Must
UM-4	Server-side access control	Must
UM-5	Admin dashboard	Must
UM-6	Account activation	Must
UM-7	First admin account	Must
UM-8	Audit of admin access	Must
UM-9	Chatbot only for logged-in users	Must

5.2 Chat and conversation history

FR-1	Customer chat with saved history	Must

Logged-in users chat with the assistant on the website. Every customer message and every assistant reply is saved permanently under the user's account in app.db. Users can list their past conversations, open one, continue it or delete it. Follow-up questions use the stored history of the current conversation as context.

Acceptance criteria

- After logging out and in again, all previous conversations are listed, newest first.
- Opening a conversation shows every message in order, with date and time.
- Continuing an old conversation adds the new messages to that same conversation.
- A deleted conversation no longer appears in the user's list (see OQ-2 for what admins see).
- The last 10 messages of the current conversation are passed to the task planner.
- A new conversation is titled automatically from its first message (first 60 characters).

FR-9	Data freshness shown	Should

Every answer that lists products shows when each category's data was last updated, taken from that agent's latest successful scrape run.

Acceptance criteria

- The reply ends with a line such as “Data last updated: clothes 10 Oct 2026 00:12”.
- If the latest scrape failed, the date of the last successful scrape is shown.

5.3 Understanding and routing

FR-2	Category detection and routing	Must

For every message, the task planner identifies which categories the customer is asking about (one to three of food, clothes and electronics). It creates one task per category, containing a cleaned sub-question and any filters the customer stated (for example maximum price, size, colour or brand), and sends each task to the matching domain agent.

Acceptance criteria

- A single-category message produces exactly one task, for the correct agent.
- A multi-category message produces one task per category mentioned, and no others.
- Stated filters appear in the task, e.g. “under 200,000” becomes max_price = 200000.
- Routing accuracy on the labelled routing test set is at least 90%.

FR-3	Unclear, out-of-scope and follow-up messages	Must

The central agent never guesses silently. It handles these cases as follows:

- **Unclear message** (e.g. “apple”): ask a short clarifying question. No agent is called.
- **Out of scope** (e.g. “What's the weather today?”): politely decline and explain that the assistant helps with food, clothes and electronics products.
- **Follow-up after a combined answer:** if the previous reply covered more than one category and the follow-up does not name a category (e.g. “show me cheaper ones”), ask which category the customer means, listing the categories from the previous reply. No agent is called until the customer answers.
- **Follow-up after a single-category answer:** route directly to that category's agent without asking.

Acceptance criteria

- “Show me cheaper ones” after a clothes + electronics reply returns a question naming both categories, and no agent_calls rows are created for that turn.
- When the customer then answers “power banks”, only the electronics agent is called.
- “Show me cheaper ones” after an electronics-only reply goes straight to the electronics agent.

FR-4	Grounded answers	Must

Each domain agent answers only from its own product database. It must never invent products, prices, availability or links. Every product in an answer includes its name, price with currency, source website and product link. When nothing matches, the agent says so and suggests relaxing the filters.

Acceptance criteria

- Every product mentioned in a reply exists in the corresponding product database.
- Every product line has a name, price, currency, source website and link.
- A query with no matches returns a clear “nothing found” message, not an invented product.
- Products marked unavailable are not suggested.

5.4 Combined answers

FR-11	Combined reply for multi-category questions	Must

When a message produces two or more tasks, the central agent sends all tasks to their agents at the same time (in parallel), waits for every agent to reply or time out (10 seconds), and returns one combined reply with a separate section for each category, in the order the customer mentioned them.

Acceptance criteria

- “A rain jacket and a power bank” returns one reply with a clothes section and an electronics section.
- Agents are called concurrently, so total time is close to the slowest agent, not the sum of all agents.
- The assistant message stores every category it covered (e.g. “clothes,electronics”) for use by FR-3.

FR-12	Partial failure handling	Must

If one agent fails, times out or finds nothing, the combined reply still contains the other categories' results and clearly states which part could not be answered and why (no results, or temporarily unavailable).

Acceptance criteria

- With the electronics agent stopped, the rain-jacket-and-power-bank question still returns the clothes section plus “I couldn't check electronics right now.”
- If every agent fails, the assistant apologises and asks the customer to try again later, and the error is logged.

FR-13	No invented content in combined replies	Must

The answer combiner may only use products returned by the agents for that message, and keeps each product's link and source website. If the LLM-written reply contains a link that no agent returned, the combiner discards it and uses the fixed template reply instead.

Acceptance criteria

- Every link in a combined reply appears in that turn's agent results.
- If the LLM is unavailable, a template reply (category headings and product lines) is returned instead of an error.

FR-14	Shared constraints across categories (stretch goal)	Could

If the customer states one constraint covering several categories (e.g. “a jacket and earbuds under 20,000 in total”), the combiner suggests combinations of one product per category whose total price stays within the limit. This is a small constraint satisfaction problem (variables: one product per category; domains: each agent's results; constraint: sum of prices ≤ budget), solvable with backtracking search. Pending OQ-4.

Acceptance criteria

- Every suggested combination respects the stated total. If none exists, the reply says so and shows the cheapest option per category.

5.5 Data collection and storage

FR-5	Website list per agent	Must

Each domain agent has its own list of shopping websites, stored in `config/websites.yaml` rather than in the code. Each entry has a site name, a base URL and the listing pages to scrape.

Acceptance criteria

- Adding a site that uses the same page layout as an existing parser takes effect at the next scrape, with no code change.
- A site with a new layout needs only a new parser in that agent's scraper.

FR-6	Separate database per agent	Must

Each domain agent owns one database (food.db, clothes.db or electronics.db) and is the only component that writes to it. Domain agents have no access to app.db or to other agents' databases; they receive only the task they must answer.

Acceptance criteria

- Each agent is constructed with only its own repository object.
- No domain agent module imports the app.db connection.

FR-7	Automatic daily refresh	Must

At 00:00 every day the scheduler starts each agent's refresh as a separate job. The agent scrapes every site in its list, cleans and normalizes the data, and upserts it into its database. Products not found in the latest successful scrape of their site are marked unavailable instead of being deleted. If a site fails, that site's previous data is kept and the failure is logged; other sites and agents continue.

Acceptance criteria

- After 00:00, each product database has new scrape_runs rows and updated scraped_at values.
- A failing site does not stop the remaining sites or agents.
- A manual command, `python -m app.scheduler.run_now <agent|all>`, runs the same refresh on demand for testing and demos.

FR-8	Common product format, without images	Must

All agents store the same core product fields (Section 8.1) plus a few domain-specific fields. **Product images are not stored**: no image URLs, no image files and no thumbnails. Scrapers read text fields only and do not download images.

Acceptance criteria

- The products tables contain no image-related columns.
- Scraper code does not request image files (checked by code review and request logs).

5.6 Logging

FR-10	Logging and traceability	Should

Every agent call made for a message is stored in agent_calls (agent, sub-question, filters, status, products found, time taken). Every scrape run is stored in that agent's scrape_runs table. Application errors are written to logs/app.log.

Acceptance criteria

- Each assistant reply produced by agents has one agent_calls row per task.
- Admins can see, for each message, which agents answered.

5.7 User management

These requirements replace the original session-only chat. They exist so that every user's chat history is kept and can be accessed by the user and by administrators.

UM-1	Registration	Must

A guest can create a customer account with name, email and password. Each email can be used only once. The password must have at least 8 characters and is stored as a bcrypt hash, never as plain text. The registration page explains that chats are saved and can be viewed by administrators.

Acceptance criteria

- Registering with an existing email returns “Email already registered”.
- The users table contains only bcrypt hashes in password_hash.
- Public registration always creates role = customer.

UM-2	Login and logout	Must

Users log in with email and password and receive a signed login token (JWT) that expires after 24 hours. The token is sent with every request. Logging out discards the token in the browser. When the token expires, the user is sent back to the login page.

Acceptance criteria

- A wrong email or password returns a generic “Invalid email or password” message.
- last_login_at is updated on each successful login.
- After 24 hours the token is rejected with 401.

UM-3	Roles	Must

There are two roles. A customer can use the chatbot and manage only their own conversations. An admin can also view every user's conversations, search users and activate or deactivate accounts.

Acceptance criteria

- A customer calling any /admin endpoint receives 403.
- An admin can open any user's conversation.

UM-4	Server-side access control	Must

Every protected endpoint checks the token, the role and, for conversations, ownership on the server. Hiding buttons in the interface does not count as protection.

Acceptance criteria

- A request with no token or an invalid token returns 401.
- A customer requesting another user's conversation receives 403.

UM-5	Admin dashboard	Must

Admins have a dashboard to list and search users (by name or email), see each user's status and last login, open a user's conversations read-only, filter them by date or category, and see which agents handled each message.

Acceptance criteria

- Searching by part of an email finds the user.
- Each assistant message in the admin view shows its categories and agent statuses from agent_calls.

UM-6	Account activation	Must

Admins can deactivate and reactivate user accounts. A deactivated user cannot log in, and any token they still hold is rejected.

Acceptance criteria

- Login by a deactivated user is refused with “Account disabled”.
- Requests with an old token from a deactivated user return 401.

UM-7	First admin account	Must

The first admin account is created with a setup script, `scripts/create_admin.py`, never through the public registration page. Further admins are created with the same script.

Acceptance criteria

- No public endpoint can create an admin or change a role.

UM-8	Audit of admin access	Must

Each time an admin opens a user's conversation list or a conversation, a row is written to admin_audit recording the admin, the user viewed, the conversation (if any) and the time.

Acceptance criteria

- Opening a user's history as admin creates an admin_audit row.

UM-9	Chatbot only for logged-in users	Must

The chatbot is shown only after login. Guests see only the login and registration pages. Opening the chat page without a valid token redirects to the login page, and the chat interface is not rendered until the server has confirmed the token (GET /me). As a backstop, the API rejects chat requests without a valid token (401).

Acceptance criteria

- A guest opening the site sees the login page, not the chatbot.
- If the token expires during a chat, the chat is hidden and the user is redirected to login; messages already sent remain saved.
- POST /chat without a token returns 401.

6. Non-functional requirements

ID	Category	Requirement	Measure / target
NFR-1	Performance	Single-category replies are returned quickly.	≤ 10 s for 95% of requests on demo hardware
NFR-2	Performance	Combined replies call agents in parallel.	≤ 15 s for 95% of requests; agent timeout 10 s
NFR-3	Reliability	A failure in one agent, website or scraper does not affect the others.	Fault-injection tests pass
NFR-4	Data freshness	Product data is refreshed daily.	Data age ≤ 24 h under normal operation
NFR-5	Security	Passwords are hashed with bcrypt and never logged or returned by the API.	No plain-text passwords in database or logs
NFR-6	Security	Login tokens are signed with a secret from .env and expire.	JWT (HS256), expiry 24 h
NFR-7	Security	Role and ownership checks run on the server for every protected endpoint.	All access-control tests pass
NFR-8	Security	Secrets (LLM key, JWT secret) are kept in .env, which is never committed.	.env in .gitignore; .env.example has placeholders only
NFR-9	Privacy	Users are told that chats are stored and can be viewed by admins.	Notice on the registration page
NFR-10	Privacy	Admin access to chat histories is recorded.	One admin_audit row per view
NFR-11	Ethical scraping	Scrapers respect robots.txt and site terms, identify themselves and limit their request rate.	≤ 1 request per second per site; descriptive User-Agent
NFR-12	Extensibility	A new product category can be added without changing existing agents.	New website list + scraper + agent class + one registration line
NFR-13	Maintainability	Code is modular and configuration-driven, with automated tests.	Layout of Section 11.2; tests for routing, combining, access control and scrapers
NFR-14	Usability	Chat works in desktop and mobile browsers; combined replies use clear category headings.	Manual walkthrough on one desktop and one mobile browser
NFR-15	Storage	Only text data is stored for products.	No image URLs or image files in any database

7. Constraints, assumptions and scope

7.1 Constraints

- The project must be finished within the module timeline by the student team.
- LLM usage should be free or low-cost; providers may impose rate limits.
- Websites may block scraping or change their layout without notice.
- SQLite single-file databases are used: suitable for a demo, not for large-scale traffic.
- The system runs on a team member's computer or one small server.

7.2 Assumptions

ID	Assumption
A-1	Domain agents do not communicate directly; the whited-out areas of the sketch are treated as having no connection (OQ-1).
A-2	Customers write their questions in English.
A-3	Prices are stored in the currency shown on each website; there is no currency conversion.
A-4	A message mentions at most three categories (one per domain agent).
A-5	The chosen websites permit scraping of their public product pages (OQ-6).
A-6	The daily refresh runs at 00:00 in the server's configured time zone.
A-7	Admins may also use the chatbot; their own chats are stored like any user's.

7.3 Out of scope

- Ordering, checkout and payment
- Real-time prices or stock levels (data is at most one day old)
- Product images (not scraped, not stored, not displayed)
- Password reset by email, and Google or Facebook login
- Guest chat without an account
- Currency conversion and cross-currency price comparison
- Mobile apps (the website works in mobile browsers)
- Personalized recommendations based on purchase history

8. Data requirements

The system has four SQLite databases: one product database per domain agent (food.db, clothes.db, electronics.db) and one application database (app.db) for users and chat history. Only the owning component reads or writes each database.

8.1 Product data (food.db, clothes.db, electronics.db)

Each product database has a `products` table with these common fields:

Field	Type	Required	Description
product_id	TEXT, primary key	Yes	SHA-1 hash of source_site + product_url
name	TEXT	Yes	Product name as shown on the website
category	TEXT	Yes	food, clothes or electronics
sub_category	TEXT	No	e.g. rice, jackets, power banks
brand	TEXT	No	Brand or manufacturer
price	REAL	Yes	Numeric price, parsed from text
currency	TEXT	Yes	Currency as shown on the site
description	TEXT	No	Short text description, trimmed to 1,000 characters
availability	TEXT	Yes	in_stock, out_of_stock, or unavailable (missing from the latest scrape)
rating	REAL	No	0–5, if the site shows ratings
product_url	TEXT, unique	Yes	Link to the product page
source_site	TEXT	Yes	Site name from websites.yaml
first_seen_at	TIMESTAMP	Yes	When the product was first scraped
scraped_at	TIMESTAMP	Yes	When the product was last seen

No images. The product tables have no image_url column, and no image files, thumbnails or other media are downloaded or stored. Scrapers ignore <img> elements (FR-8, NFR-15).

Domain-specific fields

Database	Extra fields
food.db	weight (e.g. “5 kg”), ingredients, calories (per 100 g), dietary_tags (e.g. vegetarian, vegan, gluten-free)
clothes.db	sizes (e.g. “S,M,L,XL”), colours, gender (men, women, unisex, kids), material
electronics.db	model_number, specs (JSON text, e.g. {"ram": "16GB", "storage": "512GB"}), warranty

Scrape history table: scrape_runs

Field	Type	Description
run_id	INTEGER, primary key	Auto-increment
source_site	TEXT	Site that was scraped
started_at / finished_at	TIMESTAMP	Start and end of the run
status	TEXT	ok or failed

Field	Type	Description
products_found	INTEGER	Number of products parsed
error	TEXT	Error message if the run failed

8.2 Application data (app.db)

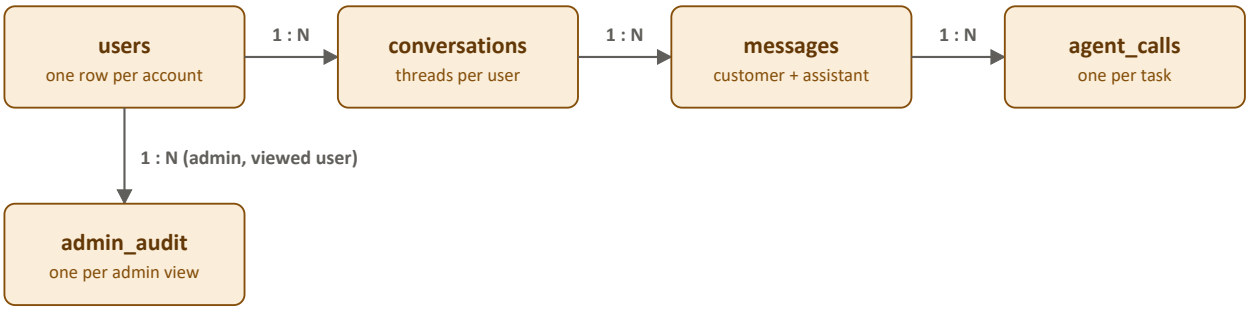


Figure 3. Relationships between app.db tables

Table: users

Field	Type	Description
user_id	INTEGER, primary key	Auto-increment
name	TEXT, not null	Display name
email	TEXT, not null, unique	Login name
password_hash	TEXT, not null	bcrypt hash; never plain text
role	TEXT, not null	customer (default) or admin
is_active	INTEGER, not null	1 = active, 0 = deactivated
created_at	TIMESTAMP	Registration time
last_login_at	TIMESTAMP, nullable	Updated on each successful login

Table: conversations

Field	Type	Description
conversation_id	INTEGER, primary key	Auto-increment
user_id	INTEGER, FK → users	Owner of the conversation
title	TEXT	First 60 characters of the first message
created_at / updated_at	TIMESTAMP	Used to sort newest first
is_deleted	INTEGER, default 0	Set when the owner deletes it (see 8.3)

Table: messages

Field	Type	Description
message_id	INTEGER, primary key	Auto-increment
conversation_id	INTEGER, FK → conversations	
sender	TEXT	customer or assistant
content	TEXT, not null	Message text
categories	TEXT, nullable	Categories an assistant reply covered, e.g. “clothes,electronics” (used by FR-3)
created_at	TIMESTAMP	

Table: agent_calls

Field	Type	Description
call_id	INTEGER, primary key	Auto-increment
message_id	INTEGER, FK → messages	The assistant reply this call produced
agent	TEXT	food, clothes or electronics
sub_question / filters	TEXT / TEXT (JSON)	The task sent to the agent
status	TEXT	ok, empty or failed
products_found	INTEGER	
latency_ms	INTEGER	Time the agent took
error	TEXT, nullable	Error message if failed

Table: admin_audit

Field	Type	Description
audit_id	INTEGER, primary key	Auto-increment
admin_id	INTEGER, FK → users	Admin who viewed the history
viewed_user_id	INTEGER, FK → users	User whose history was viewed
conversation_id	INTEGER, nullable	Conversation opened, if any
viewed_at	TIMESTAMP	

8.3 Retention and deletion

- Conversations are kept until the user deletes them.
- Until OQ-2 is decided, deletion is a **soft delete**: is_deleted is set to 1, the conversation disappears from the user's list, and admins can still see it. This lets the team switch to permanent deletion later without changing the schema.
- Deactivating a user keeps their history; it only blocks login.
- Product rows are never deleted by the daily refresh; missing products are marked unavailable.

8.4 Website list configuration

Each agent's websites are listed in `config/websites.yaml`. The URLs below are examples only; the team must choose sites that allow scraping (OQ-6). The two practice sites shown are built for scraping exercises.

```
food:
  - name: food_site_1                    # replace with a chosen grocery site
    base_url: https://www.example-grocery.com
    listing_pages: [/rice, /snacks, /beverages]
clothes:
  - name: scrapingcourse_demo            # practice store for scraping exercises
    base_url: https://www.scrapingcourse.com/ecommerce/
    listing_pages: [/]
electronics:
  - name: webscraper_test_shop           # practice store for scraping exercises
    base_url: https://webscraper.io/test-sites/e-commerce/allinone
    listing_pages: [/computers/laptops, /computers/tablets, /phones/touch]
```

9. System behaviour

9.1 Registration and login

1. A guest opens the website. Only the login and registration pages are available (UM-9).
2. **Register:** the guest submits name, email and password. The server checks that the email is unused and the password has at least 8 characters, hashes it with bcrypt and creates a user with role customer.
3. **Login:** the user submits email and password. The server checks the hash and is_active. On success it returns a JWT valid for 24 hours and updates last_login_at.
4. The browser keeps the token and sends it in the Authorization header. The chat page calls GET /me; only when that succeeds are the chatbot and history sidebar displayed.
5. On a 401 response (expired or invalid token) the browser deletes the token and redirects to the login page.
6. **Logout** deletes the token in the browser.

9.2 Single-category question

Example: “Laptops under 200,000 with 16GB RAM”.

1. The browser sends POST /chat {conversation_id, message} with the token.
2. The API verifies the token and passes the user, conversation and message to the central agent.
3. The task planner reads the last 10 messages and returns one task: {"domain": "electronics", "query": "laptop 16GB RAM", "filters": {"max_price": 200000}}.
4. The central agent calls the electronics agent with a 10-second timeout.
5. The electronics agent searches electronics.db (keywords plus filters, cheapest first, top 5) and writes a short answer using only those rows.
6. The combiner formats the reply (one section) and adds the data-freshness line.
7. The history saves the customer message, the assistant reply (categories = electronics) and one agent_calls row, and the reply is returned to the browser.

9.3 Multi-category question (combined answer)

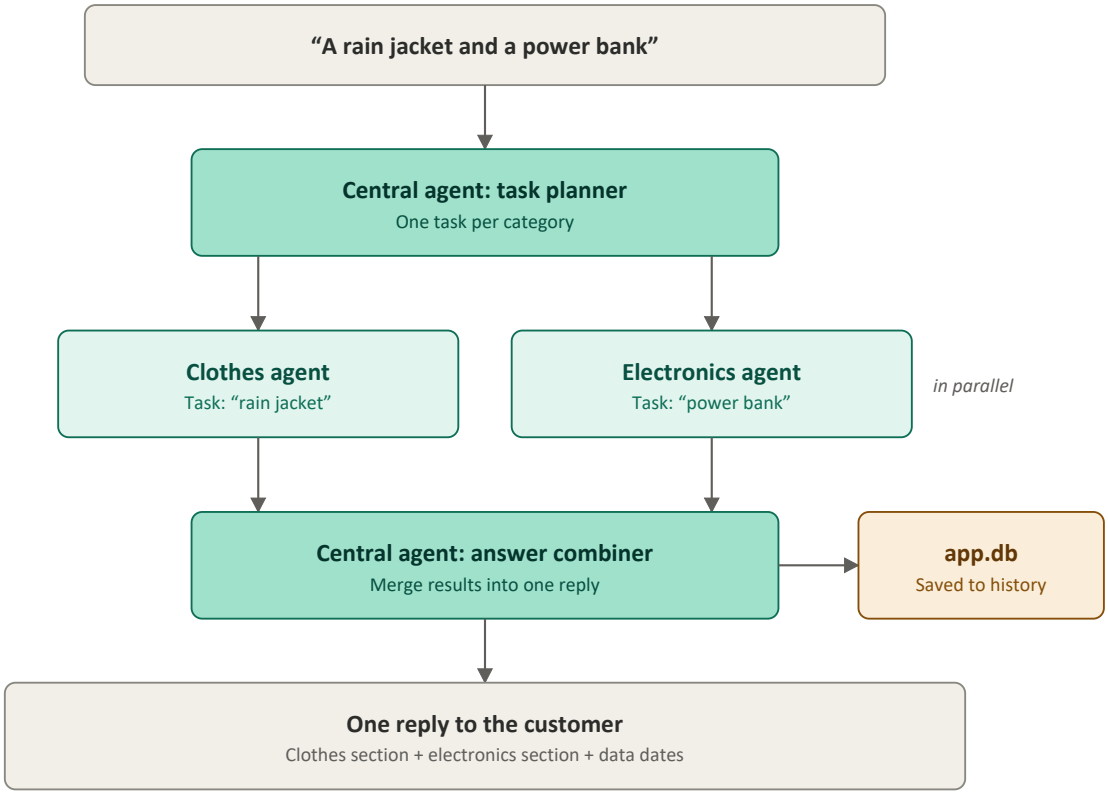


Figure 4. Combined answer for a two-category question

1. The customer asks “A rain jacket and a power bank”.
2. The task planner returns two tasks (below).
3. The central agent calls the clothes and electronics agents at the same time, each with a 10-second timeout.
4. Each agent searches its own database and returns an AgentReply (status, products, short summary, last updated).
5. The answer combiner writes one reply with a section per category, using only the returned products. If an agent failed or found nothing, its section says so (FR-12).
6. The reply is saved with categories = clothes,electronics, plus one agent_calls row per agent, and returned to the customer.

Task planner output for this message:

```
{
  "tasks": [
    {"domain": "clothes",     "query": "rain jacket", "filters": {}},
    {"domain": "electronics", "query": "power bank",  "filters": {}}
  ],
  "followup": false,
  "categories_named": ["clothes", "electronics"],
  "clarify": null,
  "out_of_scope": false
}
```

Example combined reply (prices and sites are illustrative):

```
Here's what I found for both items:

Clothes - rain jackets
  - Lightweight Rain Jacket - Rs. 6,500 - shop_a - https://...
  - Hooded Waterproof Shell - Rs. 8,900 - shop_b - https://...

Electronics - power banks
  - 10,000 mAh Power Bank - Rs. 4,200 - shop_c - https://...
  - 20,000 mAh Fast-Charge Power Bank - Rs. 7,800 - shop_d - https://...

Data last updated: clothes 10 Oct 2026 00:12, electronics 10 Oct 2026 00:20
```

9.4 Follow-up after a combined answer

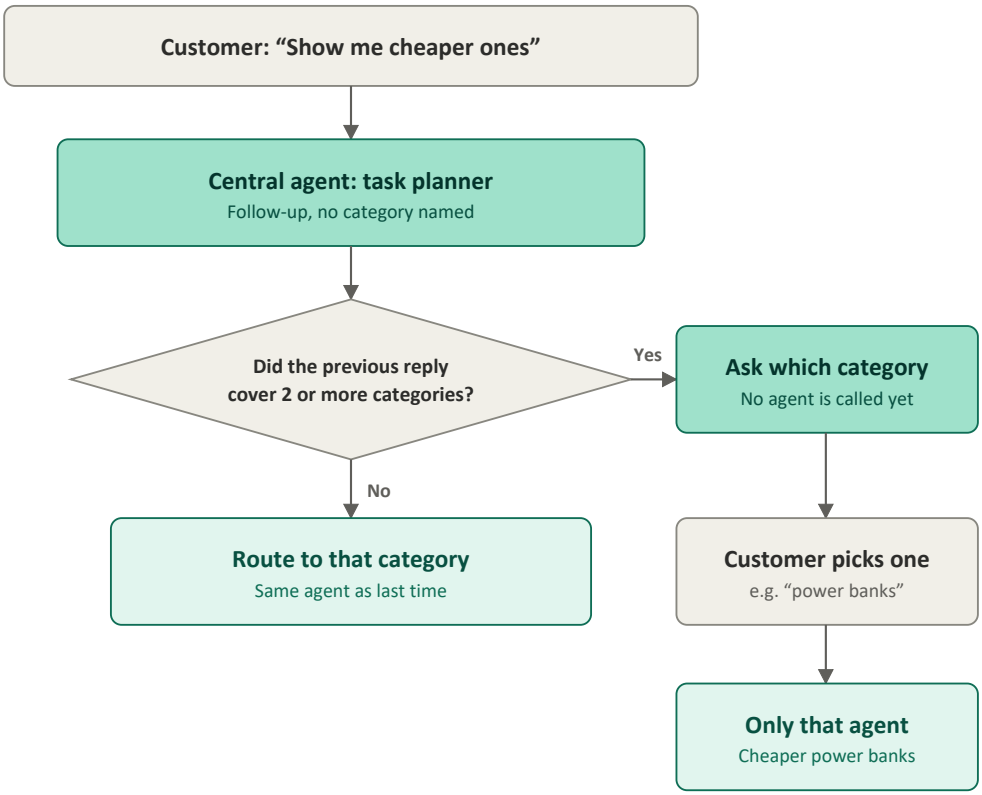


Figure 5. Handling “Show me cheaper ones”

1. The customer sends a follow-up that names no category, e.g. “Show me cheaper ones”.
2. The task planner marks it as a follow-up (followup = true, categories_named = []) and reads the categories of the previous assistant reply from the messages table.
3. If the previous reply covered two or more categories, the central agent replies with a question such as “Do you want cheaper rain jackets (clothes) or cheaper power banks (electronics)?” No agent is called.
4. The customer answers, e.g. “power banks”. The planner now creates one task for electronics, with a price limit below the cheapest power bank shown before.
5. If the previous reply covered only one category, step 3 is skipped and the follow-up goes straight to that category's agent.

9.5 Daily data refresh

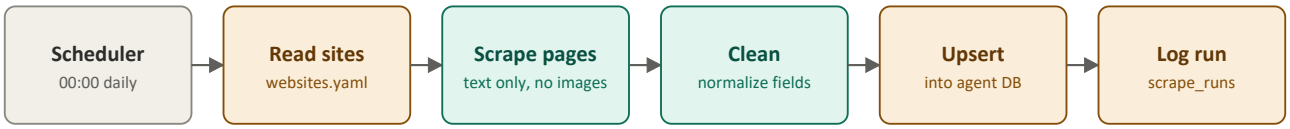


Figure 6. Daily refresh pipeline (one job per agent)

1. At 00:00 the scheduler starts one job per agent. The jobs are independent.
2. The agent reads its site list from websites.yaml.
3. For each site, the scraper checks robots.txt and fetches the listing pages at most one request per second, extracting text fields only (no images).
4. The data is cleaned and normalized: text trimmed, prices parsed to numbers, currency set, availability mapped, product_id computed.
5. Rows are upserted into the agent's products table. Products from that site that were not seen in this run are marked unavailable.
6. The run is recorded in scrape_runs with its status, product count and any error. On failure nothing is deleted, and the next site continues.

9.6 Admin reviews a user's history

1. The admin logs in; because the role is admin, a link to the admin dashboard appears.
2. The admin searches for the user by name or email (GET /admin/users?search=).
3. The admin opens the user's conversation list (GET /admin/users/{id}/conversations). An admin_audit row is written.
4. The admin opens a conversation (GET /admin/conversations/{id}) and sees the messages and, for each assistant reply, the agents called, their status and timing. Another audit row is written.
5. If needed, the admin deactivates the account (PATCH /admin/users/{id} {"is_active": false}).

9.7 Error handling

Situation	System behaviour
Message is unclear	Ask a clarifying question; no agent called (FR-3).
Message is out of scope	Politely decline and list what the assistant can help with.
Follow-up names no category after a combined answer	Ask which category; no agent called (FR-3).
One agent fails or times out (more than 10 s)	Show the other sections and say which category could not be checked (FR-12).
All agents fail	Apologise, ask the customer to try again later, log the error.
No products match	Say nothing matched and suggest relaxing filters (FR-4).
LLM unavailable while planning	Use the keyword fallback classifier (a keyword list per category).
LLM unavailable while combining	Return the template reply built from the raw results (FR-13).
A website fails during the scrape	Keep that site's previous data, log the error, continue with the next site (FR-7).
Token missing or expired	401; the browser hides the chat and redirects to login (UM-9).
Customer opens another user's conversation	403 Forbidden (UM-4).
Deactivated account tries to log in	Refused with “Account disabled” (UM-6).

10. Interface requirements

10.1 User interface pages

Page	Who	Contents
Login	Guest	Email, password, login button, link to registration. Error messages for invalid login or disabled account.
Register	Guest	Name, email, password (minimum 8 characters), privacy notice (chats are stored and may be viewed by admins), register button.
Chat (home)	Logged-in user	Sidebar: past conversations, newest first, with a New chat button and a delete option. Main area: message history and input box. Combined replies use category headings; each product shows name, price, source website and link; replies end with the data-freshness line. Not rendered until GET /me succeeds.
Admin dashboard	Admin	User list with search, status and last login; deactivate or reactivate buttons. Clicking a user lists their conversations; clicking a conversation opens a read-only view with agents and statuses for each reply.

10.2 API endpoints

Method	Endpoint	Access	Request	Response
POST	/auth/register	Guest	{name, email, password}	201 {user_id}; 409 if the email exists
POST	/auth/login	Guest	{email, password}	200 {token, role, name}; 401
POST	/auth/logout	Logged in	–	204
GET	/me	Logged in	–	{user_id, name, email, role}
POST	/chat	Logged in	{conversation_id?, message}	{conversation_id, reply, categories}
GET	/conversations	Logged in	–	[{conversation_id, title, updated_at}]
GET	/conversations/{id}	Owner	–	[{sender, content, created_at}]; 403 if not owner
DELETE	/conversations/{id}	Owner	–	204; 403 if not owner
GET	/admin/users	Admin	?search=	[{user_id, name, email, role, is_active, last_login_at}]
GET	/admin/users/{id}/conversations	Admin	–	Conversation list (audited)
GET	/admin/conversations/{id}	Admin	–	Messages + agent_calls (audited)
PATCH	/admin/users/{id}	Admin	{is_active}	200 updated user

FastAPI automatically publishes interactive documentation for these endpoints at `/docs`.

10.3 Internal agent interfaces

Task plan, produced by the task planner (JSON):

```
{
  "tasks": [{"domain": "food|clothes|electronics", "query": "...", "filters": {...}}],
  "followup": true | false,
  "categories_named": ["clothes", ...],
  "clarify": null | "question to ask the customer",
  "out_of_scope": true | false
}
```

Agent reply, returned by every domain agent to the central agent:

```
{
  "agent": "electronics",
  "status": "ok",                       // ok | empty | failed
  "products": [
    {"product_id": "a91f...", "name": "10,000 mAh Power Bank", "price": 4200,
     "currency": "Rs.", "availability": "in_stock", "source_site": "shop_c",
     "product_url": "https://...", "extras": {"model_number": "PB-10K"}}
  ],
  "summary": "Two power banks match ...",
  "last_updated": "2026-10-10T00:20:00",
  "error": null
}
```

11. Implementation guide

This section explains how to build the system. The code is a skeleton showing the structure and key logic; helper functions such as parse_price or make_id are left to the team.

11.1 Technology stack

Layer	Choice	Purpose
Language	Python 3.11+	Strong libraries for web, scraping and AI
Web framework	FastAPI + Uvicorn	REST API, automatic docs, async support for parallel agent calls
Frontend	HTML, CSS, JavaScript (fetch API)	Login, register, chat and admin pages
Agents	Plain Python classes with asyncio	Makes the multi-agent design explicit; LangGraph is an alternative (OQ-5)
LLM	Any provider behind one wrapper (e.g. Claude, OpenAI, Gemini, or a local model with Ollama)	Task planning, grounded answers, combining
Databases	SQLite + SQLAlchemy	One file per agent plus app.db; no database server needed
Scraping	requests + BeautifulSoup; Playwright for JavaScript-heavy sites	Static and dynamic pages
Scheduling	APScheduler (cron trigger)	Daily refresh at 00:00
Security	bcrypt, PyJWT	Password hashing, login tokens
Configuration	python-dotenv, PyYAML	.env secrets, websites.yaml
Testing	pytest, FastAPI TestClient	Unit, integration and access-control tests

11.2 Project structure

```
multi-agent-shop/
├── README.md
├── requirements.txt
├── .env.example                # LLM key, JWT secret, time zone (placeholders)
├── config/
│   ├── settings.py             # loads .env; paths, scrape time, timeouts
│   └── websites.yaml           # website list for each agent
├── app/
│   ├── main.py                 # FastAPI app, routers, scheduler start-up
│   ├── api/
│   │   ├── chat_routes.py      # POST /chat
│   │   └── me_routes.py        # GET /me
│   ├── auth/
│   │   ├── security.py         # bcrypt hashing, JWT create/verify
│   │   ├── dependencies.py     # get_current_user(), require_admin()
│   │   └── auth_routes.py      # /auth/register, /auth/login, /auth/logout
│   ├── users/
│   │   ├── user_repository.py
│   │   ├── audit_repository.py # admin_audit
│   │   └── admin_routes.py     # /admin/...
│   ├── history/
│   │   ├── history_repository.py # conversations, messages, agent_calls
│   │   └── history_routes.py   # /conversations
│   ├── orchestrator/
│   │   ├── central_agent.py    # plan -> parallel calls -> combine -> save
│   │   ├── task_planner.py     # splits a message into per-category tasks
│   │   ├── keyword_fallback.py # used when the LLM is unavailable
```

```
│   │   ├── answer_combiner.py    # merges agent replies into one reply
│   │   └── prompts.py
│   ├── agents/
│   │   ├── base_agent.py         # search, grounded answer, daily refresh
│   │   ├── food_agent.py
│   │   ├── clothes_agent.py
│   │   └── electronics_agent.py
│   ├── scrapers/
│   │   ├── base_scraper.py       # robots.txt check, rate limit, fetch
│   │   ├── food_scraper.py
│   │   ├── clothes_scraper.py
│   │   └── electronics_scraper.py
│   ├── db/
│   │   ├── app_db.py             # data/app.db connection and tables
│   │   ├── product_db.py         # per-agent product DB connection and tables
│   │   ├── product_repository.py # search(), upsert_many(), last_updated()
│   │   └── models.py             # Task, TaskPlan, Product, AgentReply
│   ├── scheduler/
│   │   ├── daily_jobs.py         # 00:00 -> each agent.refresh_data()
│   │   └── run_now.py            # manual refresh for testing and demos
│   └── llm/
│       └── client.py             # one wrapper for the chosen LLM provider
├── frontend/
│   ├── login.html
│   ├── register.html
│   ├── index.html                # chat + past-conversations sidebar (logged in only)
│   ├── admin.html                # user list + read-only history viewer
│   ├── css/style.css
│   └── js/
│       ├── auth.js               # login/register, token handling
│       ├── chat.js               # chat page and login guard
│       └── admin.js
├── data/                         # app.db, food.db, clothes.db, electronics.db (git-ignored)
├── logs/                         # app.log
├── scripts/
│   └── create_admin.py           # creates the first admin account
└── tests/
    ├── fixtures/                 # saved HTML pages for scraper tests
    ├── routing_set.csv           # labelled questions for routing accuracy
    ├── test_routing.py
    ├── test_combiner.py
    ├── test_auth_access.py
    ├── test_history.py
    └── test_scrapers.py
```

11.3 Core data structures (app/db/models.py)

```
from dataclasses import dataclass, field

CATEGORIES = ("food", "clothes", "electronics")

@dataclass
class Task:
    domain: str                                  # "food" | "clothes" | "electronics"
    query: str                                   # cleaned sub-question, e.g. "rain jacket"
    filters: dict = field(default_factory=dict)  # {"max_price": 8000, "size": "L"}

@dataclass
class TaskPlan:
    tasks: list[Task]                            # empty if unclear or out of scope
    followup: bool = False
    categories_named: list[str] = field(default_factory=list)
    clarify: str | None = None                   # question to ask instead of calling agents
    out_of_scope: bool = False

@dataclass
class Product:                                   # text fields only - no images
    product_id: str
    name: str
    price: float
    currency: str
    product_url: str
    source_site: str
    availability: str
    extras: dict = field(default_factory=dict)   # domain-specific fields

@dataclass
class AgentReply:
    agent: str
    status: str                                  # "ok" | "empty" | "failed"
    products: list[Product]
    summary: str                                 # short grounded answer for this category
    last_updated: str | None
    error: str | None = None
```

11.4 Domain agents (app/agents/)

All three agents share one base class. Each subclass only sets its name and maps scraped fields to the product schema.

```
class BaseAgent:
    name: str = ""                                # set by subclasses, e.g. "food"

    def __init__(self, repo: ProductRepository, scraper: BaseScraper, llm: LLMClient):
        self.repo = repo                          # this agent's own database only (FR-6)
        self.scraper = scraper
        self.llm = llm

    async def answer(self, query: str, filters: dict) -> AgentReply:
        products = self.repo.search(query, filters, limit=5)
        if not products:
            return AgentReply(self.name, "empty", [], "No matching products found.",
                              self.repo.last_updated())
        summary = await self.llm.complete(
            system=AGENT_PROMPT.format(category=self.name),
            user=build_grounded_prompt(query, products))   # the model sees only these rows
        return AgentReply(self.name, "ok", products, summary, self.repo.last_updated())

    def refresh_data(self) -> None:               # called by the scheduler every day (FR-7)
        for site in self.scraper.sites:
            run = self.repo.start_run(site.name)
            try:
                items = [self.normalize(raw) for raw in self.scraper.scrape(site)]
                self.repo.upsert_many(items)
                self.repo.mark_missing_unavailable(site.name, seen=items)
                self.repo.finish_run(run, "ok", len(items))
            except Exception as exc:              # keep previous data, log the error
                self.repo.finish_run(run, "failed", 0, error=str(exc))

    def normalize(self, raw: dict) -> Product:
        raise NotImplementedError


class FoodAgent(BaseAgent):
    name = "food"

    def normalize(self, raw: dict) -> Product:
        return Product(
            product_id=make_id(raw["source_site"], raw["url"]),
            name=clean(raw["name"]),
            price=parse_price(raw["price"]),
            currency=raw.get("currency", "Rs."),
            product_url=raw["url"],
            source_site=raw["source_site"],
            availability=map_availability(raw.get("stock")),
            extras={"weight": raw.get("weight"), "ingredients": raw.get("ingredients"),
                    "calories": raw.get("calories"), "dietary_tags": raw.get("tags")},
        )
```

Product search (app/db/product_repository.py). Keyword matching with filters is enough to start; a vector store such as ChromaDB can be added later for meaning-based search.

```
def search(self, query: str, filters: dict, limit: int = 5) -> list[Product]:
    sql = "SELECT * FROM products WHERE availability != 'unavailable'"
    args: list = []
    for word in query.split():
        sql += " AND (name LIKE ? OR description LIKE ? OR sub_category LIKE ?)"
        args += [f"%{word}%"] * 3
    if "max_price" in filters:
        sql += " AND price <= ?"
        args.append(filters["max_price"])
    if "brand" in filters:
        sql += " AND brand LIKE ?"
        args.append(f"%{filters['brand']}%")
    sql += " ORDER BY price ASC LIMIT ?"
    args.append(limit)
    return [Product.from_row(row) for row in self.conn.execute(sql, args)]
```

11.5 Central router agent (app/orchestrator/central_agent.py)

```
import asyncio

class CentralAgent:
    def __init__(self, agents: dict[str, BaseAgent], planner, combiner, history):
        self.agents = agents             # {"food": FoodAgent(...), "clothes": ..., ...}
        self.planner = planner
        self.combiner = combiner
        self.history = history

    async def handle(self, user_id: int, conversation_id: int, message: str) -> str:
        past = self.history.load(conversation_id, last_n=10)
        plan = await self.planner.plan(message, past)

        if plan.clarify or plan.out_of_scope or not plan.tasks:
            reply, results = self.planner.non_task_reply(plan), []
        else:
            calls = [self._call(task) for task in plan.tasks]
            results = await asyncio.gather(*calls)          # all agents at the same time
            reply = await self.combiner.combine(message, plan.tasks, results)

        self.history.save_turn(conversation_id, message, reply, plan, results)
        return reply

    async def _call(self, task: Task) -> AgentReply:
        agent = self.agents[task.domain]
        try:
            return await asyncio.wait_for(agent.answer(task.query, task.filters), timeout=10)
        except Exception as exc:                            # one failure never sinks the rest
            return AgentReply(task.domain, "failed", [], "", None, error=str(exc))
```

11.6 Task planner (app/orchestrator/task_planner.py)

```
class TaskPlanner:
    def __init__(self, llm: LLMClient):
        self.llm = llm

    async def plan(self, message: str, past: list[Message]) -> TaskPlan:
        last_reply = next((m for m in reversed(past) if m.sender == "assistant"), None)
        try:
            raw = await self.llm.complete_json(system=PLANNER_PROMPT,
                                               user=format_planner_input(message, past))
            plan = TaskPlan.from_json(raw)
        except Exception:
            plan = keyword_fallback(message)       # keyword list per category

        # FR-3: a follow-up that names no category
        if plan.followup and not plan.categories_named and last_reply:
            previous = last_reply.categories       # e.g. ["clothes", "electronics"]
            if len(previous) > 1:                  # combined answer -> ask, don't guess
                return TaskPlan(tasks=[], clarify=which_category_question(previous))
            plan.tasks = [t for t in plan.tasks if t.domain in previous]
        return plan
```

Planner prompt outline (app/orchestrator/prompts.py):

```
You are the task planner for a shopping assistant with three specialist agents:
food, clothes and electronics. Read the customer's message and the recent
conversation, then return JSON only, in this shape:
  {"tasks": [{"domain": ..., "query": ..., "filters": {...}}],
   "followup": bool, "categories_named": [...], "clarify": null or text,
   "out_of_scope": bool}

Rules:
- Create one task per category the message asks about (at most 3).
- Put price limits, sizes, colours and brands in "filters".
- If the message is not about food, clothes or electronics, set out_of_scope.
- If you cannot tell the category (e.g. "apple"), set clarify and return no tasks.
- If the message refers to the previous answer ("cheaper ones", "more like that"),
  set followup = true and list only the categories the customer actually named.
```

11.7 Answer combiner (app/orchestrator/answer_combiner.py)

```
class AnswerCombiner:
    def __init__(self, llm: LLMClient):
        self.llm = llm

    async def combine(self, message: str, tasks: list[Task],
                      results: list[AgentReply]) -> str:
        allowed_links = {p.product_url for r in results for p in r.products}
        try:
            text = await self.llm.complete(system=COMBINER_PROMPT,
                                           user=format_results(message, results))
            if not links_are_valid(text, allowed_links):   # FR-13: no invented products
                raise ValueError("reply contains a link no agent returned")
        except Exception:
            text = template_reply(results)                 # fixed layout, no LLM needed
        return text + freshness_footer(results)           # FR-9
```

Combiner prompt outline:

```
Write one reply to the customer using ONLY the products below.
- One section per category, with the category name as a heading,
  in the order the customer mentioned them.
- For each product give its name, price with currency, source website and link.
- If a category has status "empty", say nothing matched and suggest relaxing filters.
- If a category has status "failed", say that category could not be checked right now.
- Never add products, prices or links that are not in the input.
```

11.8 Scrapers (app/scrapers/)

```
import time
import requests
from bs4 import BeautifulSoup

class BaseScraper:
    USER_AGENT = "MultiAgentShopBot/1.0 (student project)"
    DELAY_SECONDS = 1.0                          # NFR-11: at most 1 request per second

    def __init__(self, sites: list[SiteConfig]):
        self.sites = sites
        self.session = requests.Session()
        self.session.headers["User-Agent"] = self.USER_AGENT

    def fetch(self, url: str) -> str:
        if not self.allowed_by_robots(url):        # urllib.robotparser
            raise PermissionError(f"robots.txt disallows {url}")
        time.sleep(self.DELAY_SECONDS)
        response = self.session.get(url, timeout=15)
        response.raise_for_status()
        return response.text

    def scrape(self, site: SiteConfig) -> list[dict]:
        products = []
        for page_url in self.listing_pages(site):
            soup = BeautifulSoup(self.fetch(page_url), "html.parser")
            products += [self.parse_item(card, site) for card in self.find_cards(soup)]
        return products

    # Implemented per website layout in food_scraper.py, clothes_scraper.py, ...
    def listing_pages(self, site: SiteConfig) -> list[str]: ...
    def find_cards(self, soup: BeautifulSoup) -> list: ...
    def parse_item(self, card, site: SiteConfig) -> dict: ...  # text only; never read <img>
```

Tips: save one HTML page per site in tests/fixtures/ and test parse_item against it, so layout changes are caught early. Use Playwright only for sites that build their product lists with JavaScript.

11.9 Authentication and access control (app/auth/)

```
# security.py
import datetime as dt
import bcrypt
import jwt
from config.settings import JWT_SECRET, TOKEN_HOURS

def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())

def create_token(user_id: int, role: str) -> str:
    expires = dt.datetime.now(dt.timezone.utc) + dt.timedelta(hours=TOKEN_HOURS)
    return jwt.encode({"sub": str(user_id), "role": role, "exp": expires}, JWT_SECRET,
                      algorithm="HS256")

def decode_token(token: str) -> dict:
    return jwt.decode(token, JWT_SECRET, algorithms=["HS256"])  # raises if expired or invalid
```

```
# dependencies.py
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

bearer = HTTPBearer(auto_error=False)

def get_current_user(cred=Depends(bearer)) -> User:
    if cred is None:
        raise HTTPException(401, "Login required")
    try:
        payload = decode_token(cred.credentials)
    except jwt.PyJWTError:
        raise HTTPException(401, "Session expired, please log in again")
    user = users.get(int(payload["sub"]))
    if user is None or not user.is_active:                # UM-6
        raise HTTPException(401, "Account not active")
    return user

def require_admin(user: User = Depends(get_current_user)) -> User:
    if user.role != "admin":                              # UM-3
        raise HTTPException(403, "Admins only")
    return user
```

```
# history_routes.py and admin_routes.py
@router.get("/conversations/{conversation_id}")
def get_conversation(conversation_id: int, user: User = Depends(get_current_user)):
    conv = history.get_conversation(conversation_id)
    if conv is None or conv.is_deleted:
        raise HTTPException(404)
    if conv.user_id != user.user_id:                      # UM-4: owners only
        raise HTTPException(403, "Not your conversation")
    return history.messages(conversation_id)

@router.get("/admin/users/{user_id}/conversations")
def admin_user_history(user_id: int, admin: User = Depends(require_admin)):
    audit.record(admin_id=admin.user_id, viewed_user_id=user_id)       # UM-8
    return history.list_conversations(user_id, include_deleted=True)
```

11.10 Frontend login guard (frontend/js/chat.js)

```
// UM-9: the chatbot is only shown to logged-in users
const token = localStorage.getItem("token");

async function api(path, options = {}) {
  const res = await fetch(path, {
    ...options,
    headers: { "Content-Type": "application/json",
               "Authorization": `Bearer ${token}`, ...(options.headers || {}) },
  });
  if (res.status === 401) {                    // missing, expired or invalid token
    localStorage.removeItem("token");
    window.location.replace("/login.html");
  }
  return res;
}

document.addEventListener("DOMContentLoaded", async () => {
  if (!token) return window.location.replace("/login.html");
  const me = await api("/me");
  if (me.ok) showChat(await me.json());        // chat UI stays hidden until this succeeds
});
```

Storing the token in localStorage keeps the project simple. For a production system, an HttpOnly cookie is safer against script injection.

11.11 Scheduler and application start-up

```
# app/scheduler/daily_jobs.py
from apscheduler.schedulers.background import BackgroundScheduler

def start_scheduler(agents: dict[str, BaseAgent]) -> BackgroundScheduler:
    scheduler = BackgroundScheduler(timezone=settings.TIMEZONE)
    for agent in agents.values():              # one independent job per agent
        scheduler.add_job(agent.refresh_data, "cron", hour=0, minute=0,
                          id=f"refresh_{agent.name}", max_instances=1)
    scheduler.start()
    return scheduler
```

```
# app/main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.scheduler = start_scheduler(AGENTS)
    yield
    app.state.scheduler.shutdown()

app = FastAPI(title="Multi-Agent Shopping Assistant", lifespan=lifespan)
for router in (auth_routes.router, me_routes.router, chat_routes.router,
               history_routes.router, admin_routes.router):
    app.include_router(router)
app.mount("/", StaticFiles(directory="frontend", html=True), name="frontend")
```

11.12 Configuration and running locally

`.env.example` (copy to .env and fill in; never commit .env):

```
LLM_PROVIDER=anthropic          # or openai, gemini, ollama
LLM_API_KEY=your-key-here
LLM_MODEL=your-model-name
JWT_SECRET=change-me-to-a-long-random-string
TOKEN_HOURS=24
TIMEZONE=UTC                    # set to your local time zone
AGENT_TIMEOUT_SECONDS=10
```

`requirements.txt`:

```
fastapi
uvicorn[standard]
sqlalchemy
pydantic
bcrypt
PyJWT
requests
beautifulsoup4
playwright          # only if a site needs JavaScript rendering
apscheduler
pyyaml
python-dotenv
httpx
pytest
# plus the SDK for your chosen LLM provider
```

Run the system (Windows commands shown):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env                                  (then edit the values)
python scripts/create_admin.py --email admin@example.com
python -m app.scheduler.run_now all                     (first scrape, so the DBs are not empty)
uvicorn app.main:app --reload                           (then open http://localhost:8000)
```

11.13 Build plan and team responsibilities

Phase	Work	Owner	Done when
1. Design	Approve this document, close open questions, choose websites, create the repository and skeleton.	Everyone	Spec approved; websites.yaml agreed
2. Data layer	Scraper and product DB for each agent; scrape_runs; fixture-based tests.	One person per agent	Each product DB filled by run_now
3. Domain agents	Search and grounded answer for each agent.	Same as phase 2	answer() unit tests pass
4. Users and history	Auth, app.db, history API, admin API, audit log.	One person	User management test cases pass
5. Central agent	Task planner, keyword fallback, parallel dispatch, answer combiner.	One person	Routing accuracy ≥ 90%; routing and combined test cases pass
6. Frontend	Login and register pages, chat with history sidebar, admin dashboard.	One person	UI walkthrough completed
7. Integration	Scheduler, logging, end-to-end tests, performance checks, demo script.	Everyone	All Must test cases pass; demo rehearsed

Phases 2–4 can run in parallel once phase 1 is complete. Phase 5 can start with stub agents that return fixed results, so it does not wait for phase 3.

12. Testing and acceptance

12.1 Test strategy

Level	What is tested	How
Unit	Repositories, password hashing, token handling, keyword fallback, template reply, parsers	pytest
Routing evaluation	Task planner accuracy	At least 30 labelled questions in tests/routing_set.csv; target ≥ 90%
Integration	API + central agent with stub agents; history saving; partial failure	FastAPI TestClient
Access control	401, 403 and audit behaviour for every protected endpoint	FastAPI TestClient with customer, admin and no-token clients
Scrapers	Parsing of each website layout	Saved HTML fixtures
User acceptance	Full demo flow in the browser	Manual walkthrough (Appendix A)
Operations	Daily refresh reliability	Scheduler left running for 7 days

12.2 Test cases

ID	Area	Input / action	Expected result	Req.
TC-01	Routing	“Cheapest 5kg rice?”	One task → food agent	FR-2
TC-02	Routing	“Men's denim jackets in size L?”	One task → clothes agent with filter size = L	FR-2
TC-03	Routing	“Laptops under 200,000 with 16GB RAM”	One task → electronics agent with max_price = 200000	FR-2
TC-04	Routing	“Apple”	Clarifying question (fruit or brand?); no agent called	FR-3
TC-05	Routing	“What's the weather today?”	Polite refusal explaining what the assistant can help with	FR-3
TC-06	Routing	“Show me cheaper ones” after an electronics-only answer	Goes straight to the electronics agent; cheaper products returned	FR-3
TC-07	Combined	“A rain jacket and a power bank”	2 tasks → one reply with a clothes section and an electronics section	FR-2, FR-11
TC-08	Combined	“Rice, a T-shirt and a phone charger”	3 tasks → one reply with three sections	FR-11
TC-09	Combined	Same as TC-07 with the electronics agent stopped	Clothes section plus “I couldn't check electronics right now”	FR-12, NFR-3
TC-10	Combined	“Show me cheaper ones” after a combined clothes + electronics answer	Asks which category (“cheaper rain jackets or cheaper power banks?”); no agent called	FR-3
TC-11	Combined	Customer then replies “power banks”	Only the electronics agent is called; cheaper power banks returned	FR-3
TC-12	Combined	Inspect any combined reply	Every product and link exists in that turn's agent results	FR-4, FR-13

ID	Area	Input / action	Expected result	Req.
TC-13	Combined	LLM unavailable while combining	Template reply with category headings returned, not an error	FR-13
TC-14	Data	“Laptop under 1,000”	Clear “nothing found” message with a suggestion to relax filters	FR-4
TC-15	Data	Any reply that lists products	Ends with the data-last-updated line per category	FR-9
TC-16	Data	Scheduler reaches 00:00	Each product DB has new scrape_runs rows and updated scraped_at values	FR-7, NFR-4
TC-17	Data	One website offline during the scrape	Other sites and agents update; that site's previous data kept; failure logged	FR-7, NFR-3
TC-18	Data	Add a new site (same layout) to websites.yaml	Site scraped at the next run with no code change	FR-5
TC-19	Data	Inspect product tables and the scraper request log	No image columns; no image files requested	FR-8, NFR-15
TC-20	Data	Code review of agent modules	Each domain agent receives only its own repository; no app.db access	FR-6
TC-21	Data	Any chat turn that calls agents	One agent_calls row per task, with status and time taken	FR-10
TC-22	Data	“A jacket and earbuds under 20,000 in total” (only if FR-14 is built)	Combinations with total ≤ 20,000, or a clear message that none exist	FR-14
TC-23	Users	Guest opens the website	Login page shown; chatbot not displayed	UM-9
TC-24	Users	Open the registration page	Privacy notice about stored chats and admin access is visible	UM-1, NFR-9
TC-25	Users	Customer logs in	Chatbot and past-conversations sidebar appear	UM-2, UM-9
TC-26	Users	Register with an email that already exists	“Email already registered”	UM-1
TC-27	Users	Inspect the users table after registration	password_hash holds a bcrypt hash; role = customer	UM-1, NFR-5
TC-28	Users	Login token expires while chatting	Chat hidden and redirect to login; messages already sent are still saved	UM-2, UM-9, NFR-6
TC-29	Users	Backend: POST /chat with no token	401 Unauthorized	UM-4, UM-9, NFR-7
TC-30	Users	Customer A opens customer B's conversation	403 Forbidden	UM-4, NFR-7
TC-31	Users	Customer calls GET /admin/users	403 Forbidden	UM-3, UM-4, NFR-7
TC-32	Users	Admin opens customer A's history	Allowed; admin_audit row created	UM-3, UM-8, NFR-10
TC-33	Users	Admin searches users and opens a conversation	Messages shown with agents and statuses for each reply	UM-5
TC-34	Users	Deactivated user tries to log in	Refused with “Account disabled”	UM-6
TC-35	Users	Try to create an admin through the public API	Not possible; only scripts/create_admin.py creates admins	UM-7

ID	Area	Input / action	Expected result	Req.
TC-36	Users	Customer logs out and logs back in	Past conversations listed and can be continued	FR-1
TC-37	Users	Customer deletes a conversation	Removed from the customer's list	FR-1
TC-38	Performance	20 single-category questions	95% answered in ≤ 10 s	NFR-1
TC-39	Performance	20 multi-category questions	95% answered in ≤ 15 s	NFR-2

12.3 Requirements traceability

Each requirement is linked to the test cases that verify it. Requirements without a test case are checked by review or inspection.

Requirement	Title	Verified by
FR-1	Customer chat with saved history	TC-36, TC-37
FR-2	Category detection and routing	TC-01, TC-02, TC-03, TC-07
FR-3	Unclear, out-of-scope and follow-up messages	TC-04, TC-05, TC-06, TC-10, TC-11
FR-4	Grounded answers	TC-12, TC-14
FR-5	Website list per agent	TC-18
FR-6	Separate database per agent	TC-20
FR-7	Automatic daily refresh	TC-16, TC-17
FR-8	Common product format, without images	TC-19
FR-9	Data freshness shown	TC-15
FR-10	Logging and traceability	TC-21
FR-11	Combined reply for multi-category questions	TC-07, TC-08
FR-12	Partial failure handling	TC-09
FR-13	No invented content in combined replies	TC-12, TC-13
FR-14	Shared constraints across categories (stretch goal)	TC-22
UM-1	Registration	TC-24, TC-26, TC-27
UM-2	Login and logout	TC-25, TC-28
UM-3	Roles	TC-31, TC-32
UM-4	Server-side access control	TC-29, TC-30, TC-31
UM-5	Admin dashboard	TC-33
UM-6	Account activation	TC-34
UM-7	First admin account	TC-35
UM-8	Audit of admin access	TC-32
UM-9	Chatbot only for logged-in users	TC-23, TC-25, TC-28, TC-29
NFR-1	Performance: Single-category replies are returned quickly.	TC-38
NFR-2	Performance: Combined replies call agents in parallel.	TC-39

Requirement	Title	Verified by
NFR-3	Reliability: A failure in one agent, website or scraper does not affect the others.	TC-09, TC-17
NFR-4	Data freshness: Product data is refreshed daily.	TC-16
NFR-5	Security: Passwords are hashed with bcrypt and never logged or returned by the API.	TC-27
NFR-6	Security: Login tokens are signed with a secret from .env and expire.	TC-28
NFR-7	Security: Role and ownership checks run on the server for every protected endpoint.	TC-29, TC-30, TC-31
NFR-8	Security: Secrets (LLM key, JWT secret) are kept in .env, which is never committed.	Review / inspection
NFR-9	Privacy: Users are told that chats are stored and can be viewed by admins.	TC-24
NFR-10	Privacy: Admin access to chat histories is recorded.	TC-32
NFR-11	Ethical scraping: Scrapers respect robots.txt and site terms, identify themselves and limit their request rate.	Review / inspection
NFR-12	Extensibility: A new product category can be added without changing existing agents.	Review / inspection
NFR-13	Maintainability: Code is modular and configuration-driven, with automated tests.	Review / inspection
NFR-14	Usability: Chat works in desktop and mobile browsers; combined replies use clear category headings.	Review / inspection
NFR-15	Storage: Only text data is stored for products.	TC-19

12.4 Acceptance

The system is accepted when every test case linked to a **Must** requirement passes, routing accuracy is at least 90%, and the Appendix A demo runs end to end without errors.

13. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Websites block scraping or forbid it in their terms	Medium	High	Choose permitted sites early (practice sites as fallback); respect robots.txt; keep HTML fixtures for demos.
Website layout changes break a scraper	Medium	Medium	One parser per site; fixture tests; previous data kept on failure.
LLM routes or splits a question incorrectly	Medium	Medium	Labelled routing set; JSON-only planner output; keyword fallback; ask when unsure.
LLM invents products or links	Low–medium	High	Answers generated only from returned rows; combiner checks every link (FR-13).
LLM cost or rate limits	Medium	Medium	Small, inexpensive model for planning; short prompts; option to run a local model.
Leak of personal data (chat history)	Low	High	Hashed passwords, expiring tokens, server-side checks, audit log, no secrets in the repository.
Scope creep or timeline pressure	Medium	Medium	MoSCoW priorities; Could items only after all Must tests pass.
JavaScript-heavy websites	Medium	Low	Use Playwright only for those sites.

14. Open questions

Until each question is answered, the team builds the default shown. Record the decision and update this document.

ID	Question	Default in this document	Decision
OQ-1	What was in the whited-out parts of the sketch? Should domain agents talk to each other (e.g. clothes asking electronics about smartwatches)?	No direct agent-to-agent communication	
OQ-2	When a customer deletes a conversation, is it only hidden from them (admins still see it) or removed for good?	Soft delete: hidden from the customer, visible to admins	
OQ-3	Can admins only read histories, or also delete conversations and reset passwords?	Read-only, plus deactivate and reactivate accounts	
OQ-4	For a shared budget across categories, should the reply pick combinations that fit, or just list both categories?	List both; FR-14 is a stretch goal	
OQ-5	Does the module require a specific framework (JADE, SPADE, LangGraph, CrewAI) or LLM use?	Plain Python classes with an LLM	
OQ-6	Which websites will each agent use, and do they allow scraping?	Practice sites for the demo	
OQ-7	Should the daily scrape replace old data or keep a price history?	Upsert latest values; no price history	

Appendix A. Demo script

This walkthrough exercises the main requirements in one session and can be used for the final demonstration.

Step	Who	Action or message	What it shows
1	Guest	Opens the website	Only the login page is shown (UM-9)
2	Customer	Registers, then logs in	Privacy notice; chatbot appears (UM-1, UM-2)
3	Customer	“Laptops under 200,000 with 16GB RAM”	Single-category routing with a price filter (FR-2, FR-4)
4	Customer	“A rain jacket and a power bank”	Combined reply with two sections (FR-11)
5	Customer	“Show me cheaper ones”	Asks which category (FR-3)
6	Customer	“Power banks”	Only the electronics agent is called (FR-3)
7	Customer	“What's the weather today?”	Polite out-of-scope refusal (FR-3)
8	Customer	Logs out and back in	Conversation still listed and can be continued (FR-1)
9	Admin	Logs in, searches the customer, opens the conversation	Agents used per reply; audit row written (UM-5, UM-8)
10	Team	Stops the electronics agent and repeats step 4	Clothes section plus a clear notice for electronics (FR-12)
11	Team	Runs python -m app.scheduler.run_now all	scrape_runs rows and updated freshness dates (FR-7, FR-9)